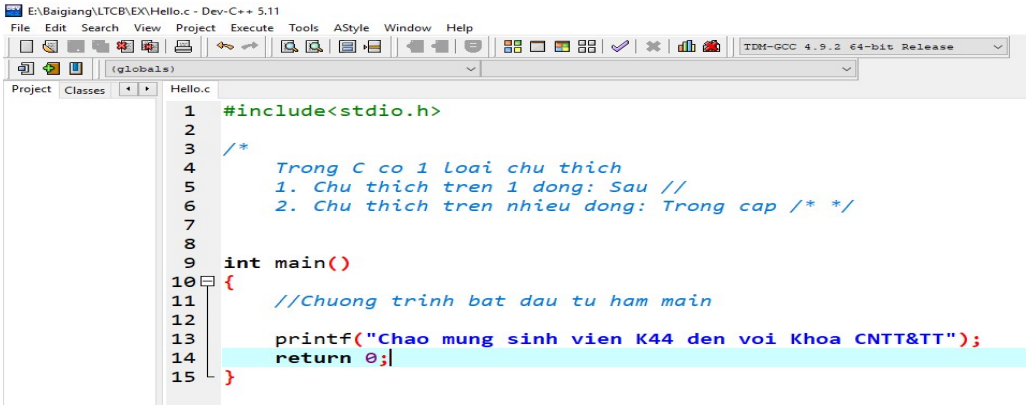
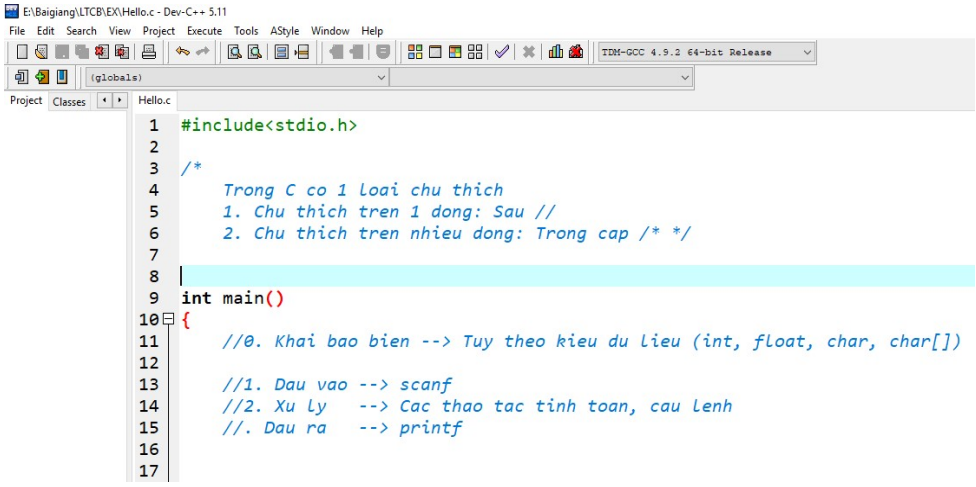
CHƯƠNG 1: XUẤT/NHẬP

Bài 1: Viết chương trình cho phép hiển thị thông điệp '`Chao mung sinh vien K44 den voi khoa CNTT&TT`' lên màn hình.

Gõ và lưu chương trình sau với tên là Hello.c



Một chương trình C bắt đầu từ hàm main(). Những chương trình C với xuất/nhập cơ bản thì chương trình có cấu trúc sau:



Phần trước return trong hàm main() là phần mà người lập trình sẽ định nghĩa chương trình của mình.

Nhìn chung một chương trình C gồm có các phần:

1. **Khai báo**. Cho phép khai báo biến tùy theo dữ liệu đầu vào, trung gian và dữ liệu đầu ra.
2. **Xử lý đầu vào**. Thường sử dụng các hàm nhập như scanf, gets để xử lý các thao tác nhập chuẩn.
3. **Tính toán**. Dùng các biểu thức, câu lệnh (rẽ nhánh, lặp) để tính kết quả.
4. **Xử lý đầu ra**. Thường sử dụng các hàm xuất như printf, puts để xử lý các thao tác xuất chuẩn.

Bài 2: Sửa lại chương trình trên cho phép nhập vào niên khóa của sinh viên. Hiển thị thông điệp chào mừng sinh viên niên khóa vừa nhập lên màn hình.

Ví dụ: `Nien khoa 45`

`Chao mung sinh vien K45 den voi khoa CNTT&TT`

Trong chương trình này, đầu vào bài toán là NienKhoa; đầu ra cho phép hiển thị 1 chuỗi với giá trị NienKhoa được đặt bên trong.

Bài 3: Viết chương trình cho phép nhập vào ngày, tháng, năm sinh của mình. Hiển thị ngày, tháng, năm sinh theo định dạng dd/mm/yyyy

Ví dụ: `12 8 1994`

→ `12/08/1994`

Bài 4: Viết chương trình cho phép nhập vào vào ngày, tháng, năm sinh của mình; sau đó nhập năm hiện hành. Tính số tuổi của mình.

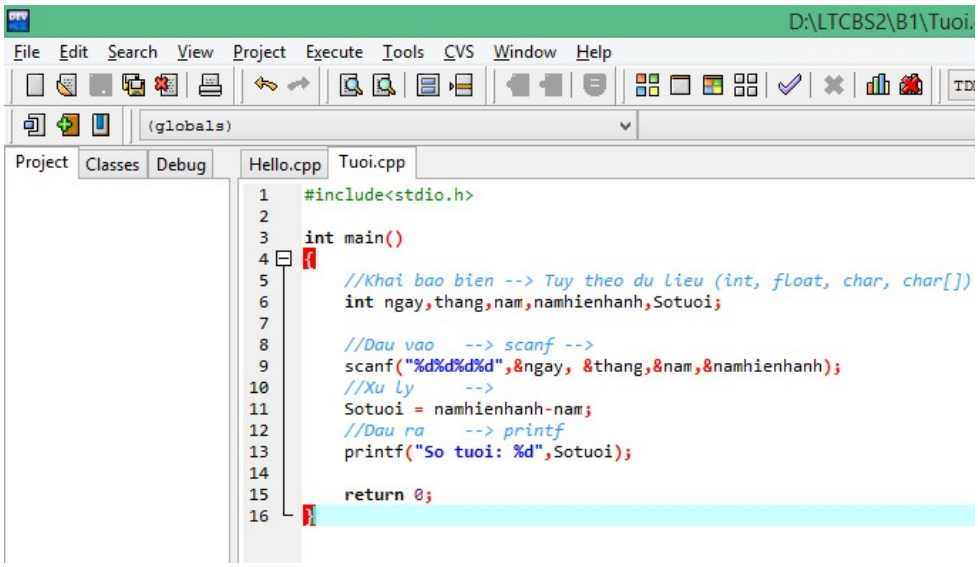
Gợi ý:

Khai báo: Ngày, tháng, năm sinh, năm hiện hành, số tuổi

Đầu vào: Ngày, tháng, năm sinh, năm hiện hành

Tính toán: Số tuổi = năm hiện hành - năm sinh

Đầu ra: Số tuổi



Bài 5: Sửa lại chương trình bài 4 sao cho không cần phải nhập năm hiện hành từ bàn phím, nhưng lấy năm hiện hành từ ngày hệ thống và hiển thị kết quả.

Bài 6: Viết chương trình cho phép nhập vào vào ngày, tháng, năm sinh của mình. Hiển thị ngày, tháng, năm sinh theo định dạng dd/mm/yy

Ví dụ: `12 8 1994`

➔ `12/08/94`

Bài 7: Môn học Lập trình căn bản được đánh giá với 1 bài thi lý thuyết và 4 bài thi thực hành. Viết chương trình cho phép nhập vào họ tên của một người cùng với điểm cuối kỳ của các bài thi của người đó (gồm bài lý thuyết và 4 bài thực hành). Hiển thị tên người đã nhập với các kết quả thi và tổng điểm của người đó. Các kết quả thi và tổng điểm được hiển thị với 2 số lẻ phần thập phân.

Để nhập chuỗi từ bàn phím ta dùng scanf ("%s", Chuoi) (hay gets(Chuoi))

Ví dụ:

```
char Ten[30];
printf ("Nhap ten: ");gets(Ten);
```

Bài 8: Yêu cầu đầu vào của bài này cũng giống như bài số 7, tuy nhiên cách tính điểm tổng cuối kỳ là điểm của bài thi lý thuyết cộng với trung bình cộng của các bài thi thực hành. Hiển thị tên người đã nhập với các kết quả thi và tổng điểm của người đó. Các kết quả thi và tổng điểm được hiển thị với 3 số lẻ phần thập phân.

Bài 9: Viết chương trình cho phép nhập vào độ dài đường kính của 1 hình tròn. Tính chu vi và diện tích của hình tròn đó.

Bài 10: Viết chương trình cho phép nhập vào độ dài 3 cạnh của 1 tam giác. Tính chu vi và diện tích của tam giác đó.

Với diện tích tam giác theo công thức Hê-rông là:

$$S = \sqrt{p(p-a)(p-b)(p-c)}$$

Trong đó a, b, c là độ dài 3 cạnh của tam giác; p là nửa chu vi.

Bài 11: Viết chương trình cho phép nhập vào giá trị nhiệt độ Fahrenheit (°F). Tính giá trị nhiệt độ Celsius (°C) tương ứng. Biết công thức chuyển đổi giữa 2 giá trị này là:

$$T_{°C} = (T_{°F} - 32) * \frac{5}{9}$$

Bài 12: Viết chương trình cho phép nhập vào giá trị nhiệt độ Celsius (°C). Tính giá trị nhiệt độ Fahrenheit (°F) tương ứng.

CHƯƠNG 2: CẤU TRÚC RẼ NHÁNH

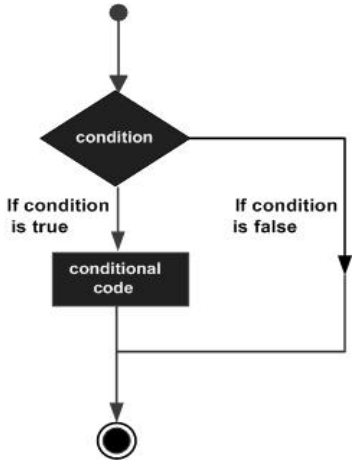
Nhắc lại Một chương trình C gồm có các phần:

1. **Khai báo**. Cho phép khai báo biến tùy theo dữ liệu đầu vào, trung gian và dữ liệu đầu ra.
2. **Xử lý đầu vào**. Thường sử dụng các hàm nhập như scanf, gets để xử lý các thao tác nhập chuẩn.
3. **Tính toán**. Dùng các biểu thức, câu lệnh (rẽ nhánh, lặp) để tính kết quả.
4. **Xử lý đầu ra**. Thường sử dụng các hàm xuất như printf, puts để xử lý các thao tác xuất chuẩn.

Chương này gồm những bài tập sử dụng câu lệnh rẽ nhánh để tính toán kết quả.

Rẽ nhánh (hay ra quyết định) là cấu trúc cho phép người lập trình xác định một hoặc nhiều điều kiện được ước lượng bởi chương trình, đi kèm với 1 câu lệnh hoặc 1 khối lệnh được thực thi khi điều kiện là đúng (!=0), và 1 câu lệnh hoặc 1 khối lệnh khác được thực thi khi điều kiện sai (tùy chọn).

Cấu trúc rẽ nhánh có dạng thức như hình sau:



Bài 1: Ở nước ta trẻ em từ 6 tuổi trở lên mới có thể vào học lớp 1. Viết chương trình cho phép nhập vào tuổi của một đứa trẻ và thông báo là em này có đủ điều kiện học lớp 1 hay không?

```
Đầu vào: Tuổi t nhập từ bàn phím

Đầu ra: Thông báo với tuổi này có vào được lớp 1 hay không?

Giải thuật

    1. Khai báo biến tuổi t

    2. Nhập tuổi t từ bàn phím

    3. if (t >= 6)        →    Thông báo: Đủ tuổi vào lớp 1

       else                   Thông báo: Không đủ tuổi vào lớp 1
```

```c
#include<stdio.h>

int main()
{
    //Khai bao bien
    int t;

    //Nhap
    printf("Nhap tuoi: ");scanf("%d",&t);

    //Xu ly
    if(t>=6) printf("Tuoi %d: Du tuoi vao lop 1\n",t);
    else printf("Tuoi %d: Khong du tuoi vao lop 1\n",t);

    return 0;
}
```

Trong chương trình này phần **Xử lý đầu ra** được tích hợp luôn vào phần **Tính toán**.

Bài 2: Môn lập trình căn bản có 5 bài kiểm tra gồm 1 bài lý thuyết và 4 bài thực hành. Tổng điểm của 5 bài kiểm tra này là kết quả của toàn môn học. Nếu tổng điểm từ 4 điểm trở lên, sinh viên được gọi là *Đạt* môn học này; ngược lại thì *Không đạt*. Viết chương trình cho phép nhập vào điểm của 5 bài kiểm tra của 1 sinh viên, thông báo là sinh viên này có đạt hay không?

Đầu vào: `Điểm lý thuyết LT, các điểm thực hành TH1, TH2, TH3, TH4`

Đầu ra: `Thông báo sinh viên có đạt hay không?`

Bài 3: Môn lập trình căn bản có 5 bài kiểm tra gồm 1 bài lý thuyết và 4 bài thực hành. Một cách tính khác là điểm trung bình của 5 bài kiểm tra này là kết quả của toàn môn học. Nếu điểm trung bình từ 4 điểm trở lên, sinh viên được gọi là *Đạt* môn học này; ngược lại thì *Không đạt*. Viết chương trình cho phép nhập vào điểm của 5 bài kiểm tra của 1 sinh viên, thông báo là sinh viên này có đạt hay không?

Đầu vào: `Điểm lý thuyết LT, các điểm thực hành TH1, TH2, TH3, TH4`

Đầu ra: `Thông báo sinh viên có đạt hay không?`

Bài 4: Cách tính điểm cuối kỳ của môn Lập trình căn bản như bài 2. Viết chương trình cho phép nhập vào điểm số của 5 bài kiểm tra của 1 sinh viên, tính điểm chữ (A, B, ..) tương ứng. Với mối tương quan giữa điểm số và điểm chữ được cho theo bảng sau:

Điểm số	**Điểm chữ**
>= 9.0	A
8.0 -< 9.0	B+
7.0 -< 8.0	B
6.0 -< 7.0	C+
5.0 -< 6.0	C
4.5 -< 5.0	D+
4.0 -< 4.5	D
< 4.0	F

Bài 5: Có 3 vận động viên (VĐV) tham gia chạy ngắn 100m để chọn người thi đấu giải quốc gia. VĐV có thành tích tốt nhất sẽ được thi đấu quốc gia, VĐV có thành tích thấp nhất sẽ xuống tập với đội trẻ; người còn lại thì vẫn ở lại đội tuyển tỉnh. Viết chương trình cho phép nhập vào thành tích thi đấu (theo giây, e.g: 12.5) của 3 VĐV. Yêu cầu:

1. Tìm thành tích thi đấu tốt nhất, thành tích thi đấu thấp nhất của 3VĐV.
2. VĐV nào được thi đấu quốc gia? VĐV nào xuống tập với đội trẻ?

Bài 6: Viết chương trình cho phép nhập từ bàn phím 2 số thực a, b biểu diễn là các hệ số của phương trình bậc 1: ax+ b =0; biện luận các trường hợp có thể có của a, b để tìm nghiệm của phương trình.

Bài 7: Viết chương trình cho phép nhập từ bàn phím 3 số thực a, b, c biểu diễn là các hệ số của phương trình bậc 2: ax^2+ bx + c =0; biện luận các trường hợp có thể có của a, b, c để tìm nghiệm của phương trình.

Bài 8: Viết chương trình nhập từ bàn phím 2 giá trị là tháng *m* và năm *y* nào đó; cho biết là vào tháng năm đó có bao nhiêu ngày.

VD: 11 2011 → Thang 11 nam 2011 co 30 ngay

2 2008 → Thang 2 nam 2008 co 29 ngay

Gợi ý: Năm nhuận là năm có ***số năm chia hết cho 400*** hoặc ***số năm chia hết cho 4 nhưng không chia hết cho 100***.

Giải thuật:

```
1. Đầu vào: tháng m, năm y
2. Xử lý
      if (tháng m có 31 ngày)          → Số ngày n = 31
      else if (tháng m có 30 ngày)     → Số ngày n = 30
      else if (m là tháng 2)
            if (Năm nhuận là đúng)     → Số ngày n = 29
            else                       → Số ngày n = 28
3. Đầu ra: Số ngày n
```

Bài 9: Viết chương trình nhập từ bàn phím 3 giá trị là dd, mm, yy biểu thị là ngày, tháng, năm nào đó; kiểm tra xem các giá trị này có biểu diễn hợp lệ 1 giá trị ngày hay không? Nêu hợp lệ, kết quả là YES, nếu không kết quả là NO.

VD: 11 3 2018 → YES

29 2 1990 → NO

Bài 10: Viết chương trình nhập từ bàn phím 3 giá trị là dd, mm, yy biểu thị là ngày, tháng, năm nào đó; tính và hiển thị giá trị ngày, tháng, năm của ngày hôm sau giá trị vừa nhập.

VD: 11 3 2018 → 12/03/2018

28 2 1990 → 01/03/1990

Bài 11: Viết chương trình nhập từ bàn phím 3 giá trị là dd, mm, yy biểu thị là ngày, tháng, năm nào đó; tính xem ngày đó là ngày thứ bao nhiêu trong năm.

Bài 12: Viết chương trình nhập từ bàn phím 3 số thực a, b, c. Hiển thị các giá trị vừa nhập theo thứ tự tăng dần mà chỉ dùng tối đa 1 biến phụ.

Bài 13: Viết chương trình nhập từ bàn phím một số nguyên dương n có 2 chữ số. Hiển thị cách đọc của số ngày lên màn hình.

VD: 12 → Muoi hai

97 → Chin muoi bay

Bài 14: Tính giá điện sinh hoạt của 1 hộ gia đình. Viết chương trình cho phép nhập từ bàn phím chỉ số điện sinh hoạt cũ (sc), và chỉ số điện mới (sm). Tính số kwh mà hộ đó tiêu thụ và số tiền mà hộ này phải đóng trong tháng, biết giá bán lẻ điện sinh hoạt được tính theo bảng sau:

Giá bán lẻ điện sinh hoạt	Giá
Bậc 1: Cho kWh từ 0-50	1.549
Bậc 2: Cho kWh từ 51 – 100	1.600
Bậc 3: Cho kWh từ 101 -200	1.858
Bậc 4: Cho kWh từ 201 -300	2.340
Bậc 5: Cho kWh từ 301 -400	2.615
Bậc 6: Cho kWh từ 401 trở lên	2.701

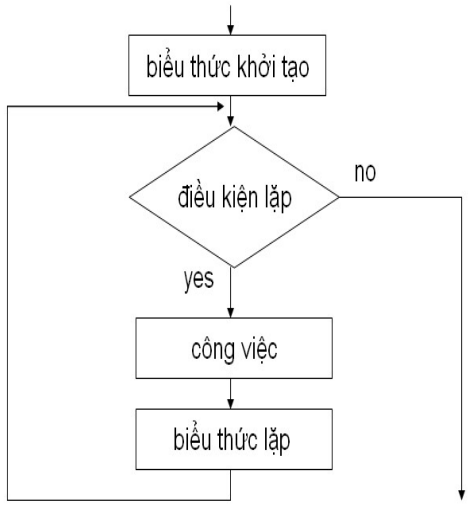
CHƯƠNG 3: VÒNG LẶP (1)

Chương này gồm những bài tập sử dụng vòng lặp để tính toán kết quả.

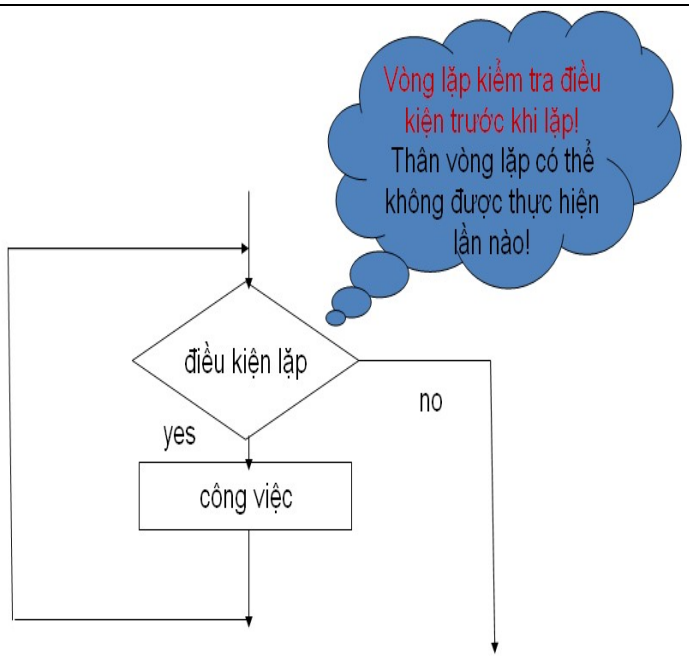
Vòng lặp được sử dụng trong một số tình huống khi ta cần thực thi một khối mã lệnh một số lần nào đó.Về cơ bản, các câu lệnh trong khối mã lệnh được thực thi tuần tự: câu lệnh đầu tiên được thực thi trước, theo sau là câu lệnh thứ hai, ...

Cấu trúc lặp là phổ biến trong hầu hết các ngôn ngữ lập trình.

STT	Vòng lặp	Cú pháp
1	**for**. Đây là một cấu trúc điều khiển cho phép ta viết một vòng lặp để thực thi một số lần xác định	**for** (*biểu thức khởi tạo; điều kiện lặp; biểu thức lặp*) **Công việc**
2	**while**. Lặp một cấu lệnh hay một khối lệnh trong khi điều kiện là đúng. Vòng lặp này kiểm tra điều kiện trước khi thực hiện phần thân.	**while** (*điều kiện lặp*) ***công việc***
3	**do … while**. Giống như vòng lặp while, tuy nhiên điều kiện được kiểm tra sau khi thực hiện phần thân.	**do** ***công việc*** **while** (*điều kiện lặp*);



*Vòng lặp **for***



*Vòng lặp **while***

*Vòng lặp **do … while***

Bài 1: Viết chương trình tính các tổng sau:

a. $S_1 = 1 + 2 + ... + n$

b. $S_2 = \frac{1}{2} + \frac{2}{3} + ... + \frac{n}{n+1}$

c. $S_3 = -1 + 2 - 3 + ... + (-1)^n n$

Với đầu vào của bài toán là 1 số nguyên n, đầu ra của bài toán là tổng S_1, S_2, S_3 được in trên 3 dòng khác nhau. Riêng tổng S_2 được in với 3 số lẻ thập phân.

Ví dụ giải thuật tính tổng S3

```
Đầu vào:  n
Đầu ra:   S3
```

Phân tích:

Tổng S_3 có thể viết lại dưới dạng $S_3 = \sum_{i=1}^{n} (-1)^i i$.

Ban đầu $S_3 = \phi$. Để tính tổng này ta dùng một vòng lặp thực hiện n lần, *ở lần thứ i ta tính giá trị của số hạng thứ i của tổng và đưa vào kết quả S_3*. Nguyên tắc tính tổng như trên gọi là nguyên tắc 'cộng tích lũy' và nó được sử dụng rất phổ biến khi lập trình.

Từ phân tích trên ta có giải thuật

```
1. Khai báo n, S3
2. Nhập số nguyên n
3. S3 = 0
   Lặp n lần từ 1 đến n
      S3 = S3 + (-1)^i*i
4. Hiển thị S3
```

```c
#include<stdio.h>
#include <math.h>

int main()
{
    //0. Declare
    int n,i,S3;
    //1. Input
    scanf("%d",&n);
    //2. Process
    S3=0;
    for(i=1;i<=n;i++)
        S3 = S3 + pow(-1,i)*i;
    //3. Output
    printf("%d\n",S3);

    return 0;
}
```

Bài 2: Viết chương trình tính n! theo công thức: `n! = 1*2* ... *n`, với đầu vào là 1 số nguyên n, đầu ra là giá trị của n!.

Bài 3: Viết chương trình tính số hạng thứ n của dãy số Fibonaci theo công thức truy hồi sau:

$$Fn = \begin{cases} 1, \text{nếu } n = 0 \text{ hoặc } n = 1 \\ Fn-2 + Fn-1, \text{nếu } n >= 2 \end{cases}$$

Với đầu vào là 1 số nguyên dương n, đầu ra là số hạng thứ n của dãy Fibonaci Fn

Bài 4: Viết chương trình liệt kê (n+1) số hạng đầu tiên của dãy Fibonaci. Đầu vào của bài toán là số nguyên dương n, đầu ra là dãy Fibonaci gồm các số hạng tương ứng, mỗi số hạng cách nhau khoảng trắng.

Ví dụ: 5

1 1 2 3 5 8

Bài 5: Tính x^n theo công thức sau: $x^n = \underbrace{x*x*...*x}_{n \text{ lần}}$. Với số thực x và số nguyên dương n đầu vào, đầu ra là kết quả x^n được hiển thị với 4 số lẻ thập phân.

Bài 6: Hiển thị các giá trị từ 1 đến m*n trong 1 ma trận m dòng, n cột từ trái sang phải, từ trên xuống dưới. Với đầu vào là 2 số nguyên m, n; đầu ra là ma trận kết quả.

Ví dụ: 2 3

1 2 3

4 5 6

Bài 7: Hiển thị các giá trị từ 1 đến m*n trong 1 ma trận m dòng, n cột từ trên xuống dưới, từ trái sang phải. Với đầu vào là 2 số nguyên m, n; đầu ra là ma trận kết quả.

Ví dụ:

2	3	
1	3	5
2	4	6

Bài 8. Tính tổng thời gian thi đấu của n thành viên trong 1 đội đua xe đạp ở Cúp Truyền Hình. Chương trình cho phép nhập số VĐV trong đội (n), rồi nhập thời gian từng VĐV. Sau đó cho biết:

a. Tổng thời gian thi đấu của đội là bao nhiêu?

b. Thời gian thi đấu trung bình của cả đội.

Đầu vào: Số thành viên n, thời gian từng thành viên ti

Đầu ra: Tổng thời gia T, thời gian trung bình TB

Giải thuật

```
1.   Khai báo biến n, ti, T, TB
2.   Nhập số thành viên n
3.   T=0
     Lặp n lần từ VĐV 1 đến VĐV n
          - Nhập thời gian của VĐV i: ti
          - T = T + ti
     Tính giá trị trung bình TB = T/n
4.   Hiển thị T, TB
```

Bài 9. Tính điểm trung bình các môn của SV trong 1 học kỳ. Chương trình cho phép nhập vào số môn trong học kỳ (n), rồi nhập điểm từng môn (là điểm số từ 0.0f đến 10.0f). Sau đó cho biết:

a. Điểm trung bình của sinh viên đó

b. Điểm lớn nhất của sinh viên đó. Đó là môn nào?

c. Điểm nhỏ nhất của sinh viên đó. Đó là môn nào?

Giải thuật tìm giá trị lớn nhất của n giá trị $t_1, t_2, ..., t_n$ là:

1. Gọi t_{Max} là giá trị cần tìm. Ban đầu $t_{Max} = t_1$
2. Lặp n-1 lần từ 2 đến n
   - Ở lần lặp thứ i: **if** ($t_{Max} < t_i$) $t_{Max} = t_i$
3. Hiển thị t_{Max}

Bài 10. Viết chương trình tìm số nguyên dương k nhỏ nhất sao cho $2^k > n$ với n là một số nguyên dương nhập từ bàn phím. Đầu vào của bài toán là số nguyên dương n, đầu ra là số nguyên dương k thỏa điều kiện.

VD: n = 5 → k=3

Bài 11. Viết chương trình tìm số nguyên dương n nhỏ nhất sao cho $1+\frac{1}{2}+\frac{1}{3}+...+\frac{1}{n}>S$, với S là số thực nhập từ bàn phím. Đầu vào của bài toán là số thực S, đầu ra là số nguyên dương n thỏa điều kiện.

Chương 4: VÒNG LẶP (2)

Bài 1: Viết chương trình in ra màn hình các số nguyên từ 1 đến 100 sao cho cứ 10 số thì xuống dòng.

Gợi ý: Cho biến i chạy từ 1 → 100. In ra màn hình i và kiểm tra: nếu i % 10==0 thì printf("\n").

Bài 2: Viết chương trình in ra màn hình bảng cửu chương.

Gợi ý: Dùng 2 vòng lặp **for** lồng nhau: i là số bảng cửu chương (2...9), j là số thứ tự trong từng bảng cửu chương (1...10).

Bài 3: Viết chương trình tính các tổng sau:

a. S0 = n! = 1*2*...*n

b. S1 = $1+\frac{1}{2}+...+\frac{1}{n}$

c. S2 = $1+\frac{1}{2!}+...+\frac{1}{n!}$

d. S3= $1+x+\frac{x^2}{2!}+...+(-1)^n\frac{x^n}{n!}$

Với n là số nguyên dương nhập từ bàn phím. Riêng *câu d,* số thực x và số nguyên dương n là các yếu tố đầu vào.

Bài 4: Viết chương trình tìm các số có 3 chữ số $\overline{abc}$ sao cho: $\overline{abc} = a^3 + b^3 + c^3$.

Ý tưởng: Dùng phương pháp vét cạn. Ta biết rằng: a có thể có giá trị từ 1→9 (vì a là số hàng trăm), b,c có thể có giá trị từ 0→9. Do đó, ta sẽ dùng 3 vòng lặp for lồng nhau để duyệt qua tất cả các trường hợp của a,b,c. Ứng với mỗi bộ $\overline{abc}$, ta sẽ kiểm tra: Nếu 100.a + 10.b + c = $a^3 + b^3 + c^3$ thì in ra bộ $\overline{abc}$ đó.

Bài 5: Viết chương trình để tìm lời giải cho bài toán sau:

Trong giỏ vừa thỏ vừa gà,

Một trăm cái cẳng bốn ba cái đầu.

Hỏi có mấy gà mấy thỏ?

Bài 6: Viết chương trình để tìm lời giải cho bài toán sau:

Trăm trâu trăm bó cỏ

Bó lại cho tròn

Trâu đứng ăn năm

Trâu nằm ăn ba

Năm trâu nghé ăn một.

Hỏi có bao nhiêu trâu đứng, trâu nằm, trâu nghé?

Bài 7: Viết chương trình nhập vào các số nguyên từ bàn phím cho đến khi nào gặp số nguyên tố thì kết thúc nhập. Tính tổng các số chẵn và trung bình cộng các số lẻ.

Gợi ý: Dùng vòng lặp **do** {...}**while**(!nto); để nhập. Trong đó, nto là biến kiểu **int** để kiểm tra số được nhập vào có phải là số nguyên tố hay không.

Bài 8: Viết chương trình nhập vào một số nguyên dương. Hãy thông báo lên màn hình số đó có bao nhiêu chữ số và tổng các chữ số của số đó.

Gợi ý: Dùng vòng lặp **while**:

Trong khi N>0 thì: lấy ra chữ số cuối cùng của N để tính bằng phép toán % 10, sau đó bỏ bớt đi chữ số cuối cùng của N bằng phép toán /10.

Bài 9: Viết chương trình in ra màn hình tất cả các số nguyên tố từ 2 đến N. Với N được nhập từ bàn phím.

Bài 10: Viết chương trình phân tích một số ra thừa số nguyên tố. Ví dụ: N=100 sẽ in ra màn hình:

Ví dụ: N= 100 → 2 2 5 5

Bài 11: Viết chương trình nhập vào số nguyên N. In ra màn hình tất cả các ước số của N.

Ý tưởng: Cho biến i chạy từ 1 tới N. Nếu N % i==0 thì viết i ra màn hình.

Bài 12: Viết chương trình tìm ước số chung lớn nhất (USCLN) và bội số chung nhỏ nhất (BSCNN) của 2 số nguyên dương m, n

Ý tưởng:

a. Tìm USCLN.

- Nếu a chia hết cho b thì USCLN = b
- Ngược lại, gọi r = a%b thì thay vì tìm USCLN(a,b) thì ta tìm USCLN(b, r)

b. BCNN(a, b) = a*b/USCLN

Bài 13: Số hoàn thiện là số tự nhiên có tổng các ước của nó (không kể chính nó) bằng chính nó. Viết chương trình kiểm tra xem một số được nhập vào từ bàn phím có phải là số hoàn thiện hay không?

Ví dụ: 6, 28 là các số hoàn thiện.

Gợi ý: - Tính tổng các ước số của N: từ 1 → N/2 lưu vào biến S.

- Nếu S==N thì N là số hoàn thiện.

Bài 14: Viết chương trình tính số Pi với độ chính xác Epsilon, biết:

$\frac{Pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \ldots$, với Epsilon là hằng số đầu vào rất nhỏ

Ta thấy rằng, mẫu số là các số lẻ có qui luật: 2*i+1 với i=1,...,n. Do đó ta dùng i làm biến chạy.

Vì tính số Pi với độ chính xác Epsilon nên không biết trước được cụ thể số lần lặp, do đó ta phải dùng vòng lặp while hoặc do{}while. Có nghĩa là phải lặp trong khi t= $\frac{4}{2i+1}$ > Epsilon.

Bài 15: Viết chương trình tính gần đúng căn bậc hai của một số dương a theo phương pháp Newton:

Trước hết cho x_0= $\frac{1+a}{2}$ sau đó là công thức truy hồi để tính các giá trị tiếp theo:

$$x_k = \frac{x_{k-1} + \frac{a}{x_{k-1}}}{2}$$

Cứ tính như vậy đến khi $\left|\frac{x_n + x_{n-1}}{x_{n-1}}\right| < Epsilon$ thì $\sqrt{a}$ =x_n

Trong đó Epsilon là một hằng số cho trước làm độ chính xác.

Bài 16: Viết chương trình tính gần đúng căn bậc n của một số dương a theo phương pháp Newton :

Trước hết cho x_0= $\frac{a}{n}$ sau đó là công thức truy hồi để tính các giá trị tiếp theo:

$$x_k = \left|\frac{(n-1)x_{k-1}^n + a}{nx_{k-1}^{n-1}}\right|$$

Cứ tính như vậy đến khi $\left|a - x_m^n\right| < Epsilon$ thì x_m=$\sqrt[n]{a}$.

Trong đó Epsilon là một hằng số cho trước làm độ chính xác. Nếu a < 0 và n chẵn thì không tồn tại căn.

Chương 5: HÀM

1. Viết hàm để trả về ký tự thường của ký tự ch.

Gợi ý: Trong bảng mã ASCII, số thứ tự của ký tự hoa nhỏ hơn ký tự thường tương ứng 32. Đầu vào của hàm là ký tự ch, đầu ra là dạng viết hoa của ch. Do đó hàm này có khuôn mẫu sau: char LowCase(char ch).

```
#include<stdio.h>

char LowCase(char ch)
{
    if(ch>='A' && ch <='Z') return ch+32;
    else return ch;
}

int main()
{
    char c;
    printf("Input a character: ");scanf("%c",&c);
    printf("The lowcase of %c is %c\n",c,LowCase(c));
    return 0;
}
```

2. Viết hàm để tính X^n.

Gợi ý: Đầu vào của hàm là 1 số thực X và 1 số nguyên dương n; đầu ra của hàm là 1 số thực biểu diễn X^n. Do đó hàm này có khuôn mẫu sau: float XMuN(float X, int n)

```
#include<stdio.h>

float XMuN(float X, int n)
{
    int i;
    float f=1.0f;
    for(i=1;i<=n;i++)
        f = f*X;
    return f;
}

int main()
{
    int n;
    float X;
    scanf("%f%d",&X,&n);
    printf("%.4f",XMuN(X,n));
    return 0;
}
```

3. Viết hàm để phân tích số nguyên dương n ra thừa số nguyên tố. Viết hàm main() cho phép nhập vào số nguyên dương N và gọi thực thi hàm trên.

Gợi ý: Đầu vào của hàm là 1 số nguyên dương n; hàm không trả về giá trị nào mà chỉ in ra dãy các thừa số nguyên tố của n. Do đó hàm này có kiểu trả về là void như khuôn mẫu sau: void PrimeFactorization (int n).

Một ý tưởng của hàm này xét từ số nguyên tố đầu tiên i=2. Nếu n vẫn còn chia hết cho i thì xuất i là 1 thừa số nguyên tố của n, ngược lại tăng i lên 1 đơn vị. Quá trình trên được lặp đi lặp lại cho tới chừng nào n không thể chia cho i được nữa (tức n=1).

4. Viết hàm để hoán đổi nội dung của 2 biến số thực a và b.

```
#include<stdio.h>

void Swap(float *a, float *b)
{
    float t=*a;
    *a=*b;
    *b=t;
}

int main()
{
    float m,n;
    m=20;
    n=30;
    printf("Before calling function m=%.3f; n=%.3f\n",m,n);
    Swap(&m,&n);
    printf("After calling function m=%.3f; n=%.3f\n",m,n);

    return 0;
}
```

- Thực thi chương trình trên và ghi nhận kết quả.
- Hàm Swap nếu ta bỏ các dấu * trước a, b. Chạy lại chương trình và ghi nhận kết quả.
- Ta có thể kết luận gì về 2 phiên bản khác nhau của hàm Swap.

5. Viết hàm int Perfect(int n); để kiểm tra số nguyên n có phải là số hoàn thiện hay không? Nếu n là số hoàn thiện, kết quả trả về của hàm là 1, ngược lại kết quả của hàm là 0. Viết hàm main() cho phép nhập vào 1 số nguyên n, gọi hàm trên và hiển thị kết quả là YES nếu n là số hoàn thiện, ngược lại hiển thị NO.

6. Viết hàm tìm bội số chung lớn nhất của 2 số nguyên dương a, b. Áp dụng hàm này trong hàm main() để tìm BSCNN của 1 số nhập từ bàn phím.

7. Viết hàm để tối giản phân số $\frac{a}{b}$, với a, b là các số nguyên. Trong hàm main(), nhập vào 2 số m, n biểu diễn phân số $\frac{m}{n}$, hiển thị phân số tối giản của phân số này.

8. Viết hàm đệ quy để tính:

a. $S1 = 1+\frac{1}{2}+...+\frac{1}{n}$

b. $S2 = 1 + \sin(x) + \sin^2(x) + \ldots + \sin^n(x)$

9. Viết hàm đệ quy để tính C_n^k, biết:

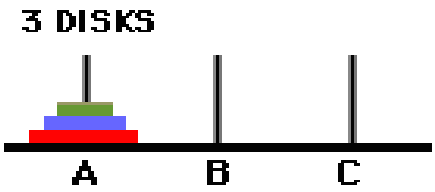
$$C_n^k = \begin{cases} 1, \text{nếu } k == n \text{ hoặc } k == 0 \\ C_{n-1}^{k-1} + C_{n-1}^{k}, \text{ngược lại} \end{cases}$$

10. Cho m, n nguyên dương. Lập hàm đệ quy để tính hàm Ackermann được cho như sau:

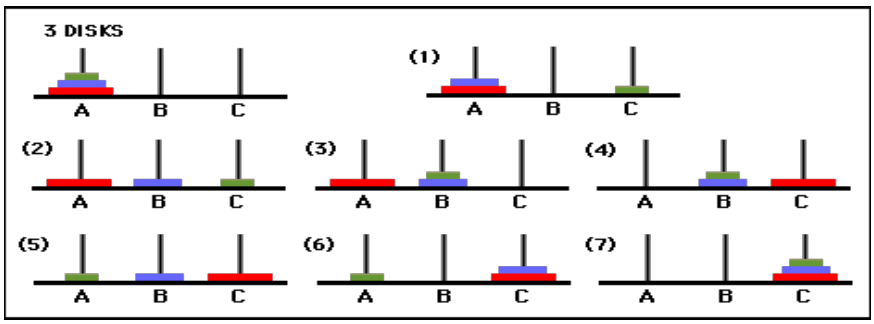
$$A(m,n) = \begin{cases} n+1, m == 0 \\ A(m-1,1), \text{nếu } m > 0 \text{ AND } n == 0 \\ A(m-1, A(m,n-1)), \text{ nếu } m > 0 \text{ AND } n > 0 \end{cases}$$

11. Viết hàm để in ra màn hình số đảo ngược của một số nguyên cho trước theo 2 cách: đệ quy và không đệ quy.

12. Bài toán Tháp Hà Nội: Có 3 cây cọc và một chồng n đĩa có kích thước từ nhỏ đến lớn (không trùng kích thước). Các đĩa đều có lỗ bên trong để chồng vào các cây cọc. Ban đầu n đĩa được chồng vào cây cọc thứ nhất, đĩa lớn nhất ở dưới, và kích thước đĩa giảm dần từ dưới lên:



- Nhiệm vụ của chúng ta là phải di chuyển n cái đĩa từ cọc A sang cọc C với ràng buộc là: Có một cái cọc trung gian là B, mỗi lần chỉ được di chuyển 1 cái đĩa từ một cọc đến cọc khác và đĩa to hơn không được xếp lên trên đĩa nhỏ hơn.
- Ví dụ dưới đây cho thấy cách di chuyển 3 cái đĩa từ A sang C:



Chương 6: BÀI TẬP TỔNG HỢP 1

Bài 1: Viết chương trình nhập vào số nguyên chỉ số đo độ của một góc và cho biết nó thuộc góc phần tư thứ mấy trên đường tròn lượng giác.

Gợi ý

- Nhập vào số đo lượng giác bất kỳ (x > 0).
- Xác định cung lượng giác theo điều kiện dưới đây:
  - Góc phần tư thứ I: (360*k) ≤ x < (90 + 360*k)
  - Góc phần tư thứ II: (90 + 360*k) ≤ x < (180 + 360*k)
  - Góc phần tư thứ III: (180 + 360*k) ≤ x < (270 + 360*k)
  - Góc phần tư thứ III: (270 + 360*k) ≤ x < 360*(k+1)

Bài 2: Bảng giá cước taxi 4 chỗ được cho trong bảng sau:

Chủng loại xe	Giá mở cửa (VNĐ)	Trong phạm vi 30km (VNĐ)	Từ km thứ 31 trở đi (VNĐ)
Toyota Vios (5 chỗ)	11.000/500m	14.500/1km	11.600/1km

Viết chương trình cho phép nhập từ bàn phím số km đã đi. Tính số tiền phải trả và hiển thị lên màn hình.

Bài 3: Viết chương trình tính sin(x) với độ chính xác Epsilon, biết:

$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} + \ldots + (-1)^n \frac{x^{2n+1}}{2n+1}$, với Epsilon là hằng số đầu vào rất nhỏ và x là giá trị nhập từ bàn phím.

Bài 4: Viết chương trình tính cos(x) với độ chính xác Epsilon, biết:

$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} + \ldots + (-1)^n \frac{x^{2n}}{(2n)!}$, với Epsilon là hằng số đầu vào rất nhỏ và x là giá trị nhập từ bàn phím.

Tổng này được tính với n đủ lớn sao cho bất đẳng thức: $\left|\frac{x^{2n}}{(2n)!}\right| \le \text{Epsilon}$ thỏa mãn.

Bài 5: Số đối xứng hay còn gọi là "palindrome". Ví dụ: 123321 là số đối xứng, 12012 không phải số đối xứng. Viết chương trình cho phép nhập vào 1 số nguyên dương n, cho biết n có là số palindrome hay không? Nếu có hiển thị 'YES', ngược lại 'NO'.

Bài 6: Viết chương trình cho phép nhập từ bàn phím 1 số nguyên n 4 bytes. Kiểm tra n có dạng 2^k không? Nếu có hiển thị 'YES', ngược lại 'NO'

Ví dụ: n = 16 → YES

n = 25 → NO

Bài 7: Biết rằng

$$n!! = \begin{cases} 1*3*5*\ldots*n, \text{nếu n chẵn} \\ 2*4*6*\ldots*n, \text{nếu n lẻ} \end{cases}$$

Viết chương trình tính n!! bằng 2 cách:

a. Vòng lặp

b. Đệ quy

Bài 8: Tam giác Pascal là một bảng số, trong đó dòng thứ 0 bằng 1, mỗi một số hạng của dòng thứ n+1 là một tổ hợp chập k của n ($C_n^k = \frac{n!}{k!(n-k)!}$)

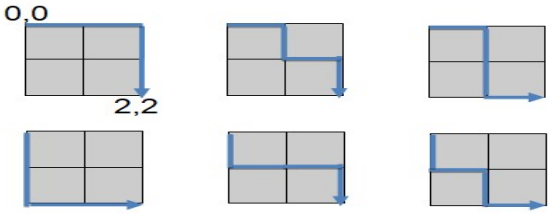
Ví dụ tam giác Pascal có 7 dòng (từ dòng 0 đến dòng 6)

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

Viết chương trình in lên màn hình tam giác Pascal có n dòng, với n nhập từ bàn phím.

Bài 9: Cho 1 lưới hình chữ nhật. Có bao nhiêu đường có thể đi từ tọa độ (0, 0) đến (x, y) trong lưới?

Ví dụ trong hình dưới biểu diễn một lưới kích thước 2x2. Chúng ta bắt đầu từ điểm có toạ độ (0, 0) và tìm đường đi đến điểm có tọa độ (2, 2) - từ điểm trên cùng bên trái đến điểm dưới cùng bên phải. Tại mỗi điểm, chỉ có thể có 2 bước đi: đi xuống hoặc đi sang phải. Với lưới 2x2 thì ta có 6 con đường đi có thể từ (0, 0) đến (2, 2).



Với điểm dưới cùng bên phải là (x,y), x>=0, y>=0 (x, y được nhập từ bàn phím), hãy viết chương trình để tính ra có bao nhiêu con đường đi có thể từ (0,0) đến nó.

Chương 7: MẢNG

Bài 1. Người ta cần quản lý điểm thi môn Lập trình căn bản của các sinh viên trong lớp bằng một mảng với mỗi phần tử trong mảng là 1 số thực (từ 0.0f đến 10.0f). Anh (Chị) hãy:

a. Viết các hàm nhập và hiển thị một mảng gồm n điểm thi của các sinh viên trong lớp.

b. Viết hàm main() cho phép gọi các hàm trên để nhập và hiển thị các điểm thi vừa nhập.

c. Một số tiêu chí thống kê để đánh giá sức học môn Lập trình căn bản của cả lớp là điểm lớn nhất, điểm nhỏ nhất và điểm trung bình trên tất cả thành viên trong lớp. Viết 3 hàm tính những giá trị này.

d. Liệt kê danh sách sinh viên với điểm số và điểm chữ tương ứng. Điểm chữ được tính theo điểm số theo công thức:

```
Diem >= 9.0            →    A
8.0 <= Diem < 9.0      →    B+
7.0 <= Diem < 8.0      →    B
6.5 <= Diem < 7.0      →    C+
6.0 <= Diem < 6.5      →    C
5.0 <= Diem < 6.0      →    D+
4.0 <= Diem < 5.0      →    D
< 4.0                  →    F
```

e. Viết hàm main() để gọi các hàm ở câu c. và d.

Bài 2: Tính tổng thời gian thi đấu của n thành viên trong 1 đội đua xe đạp ở Cúp Truyền Hình. Chương trình cho phép nhập số VĐV trong đội (n), rồi nhập thời gian từng VĐV. Sau đó cho biết:

a. Tổng thời gian thi đấu của đội là bao nhiêu?

b. Thời gian thi đấu trung bình của cả đội.

c. Vận động viên có thời gian thi đấu tốt nhất ? Thời gian tốt nhất là bao nhiêu?

d. Vận động viên có thời gian thi đấu kém nhất? Thời gian kém nhất là bao nhiêu?

Bài 3: Viết chương trình nhập vào một mảng số tự nhiên gồm n phần tử. Hãy xuất ra màn hình:

- Dòng 1 : gồm các số lẻ, tổng cộng có bao nhiêu số lẻ.
- Dòng 2 : gồm các số chẵn, tổng cộng có bao nhiêu số chẵn.
- Dòng 3 : gồm các số nguyên tố.
- Dòng 4 : gồm các số không phải là số nguyên tố.

Bài 4: Viết chương trình nhập vào một ma trận (mảng hai chiều) các số nguyên, gồm m dòng, n cột. In ma trận đó lên màn hình.

Nhập một số nguyên khác vào và xét xem có phần tử nào của ma trận trùng với số này không ? Ở vị trí nào ? Có bao nhiêu phần tử ?

Nhập tiếp một ma trận thứ hai cũng có m dòng, n cột. Tính ma trận tổng của 2 ma trận đã nhập và hiển thị kết quả lên màn hình. Biết tổng 2 ma trận A và B được tính theo công thức: $C_{ij} = A_{ij} + B_{ij}$ ($\forall i = \overline{0..m}$, $\forall j = \overline{0..n}$)

Chương 8: CHUỖI & CON TRỎ

Chương này gồm các bài tập về việc sử dụng chuỗi ký tự trong C. Việc sử dụng này cũng bao hàm một số tác vụ có sử dụng con trỏ.

Một số thao tác cơ bản

- Để sử dụng các hàm thao tác trên chuỗi ký tự, thư viện string.h được sử dụng. Một số thao tác cấp phát có thể phải sử dụng thư viện malloc.h

- Để nhập chuỗi từ bàn phím, hàm gets() được sử dụng:

```
char str[50];
gets(str);
```

- Để hiển thị chuỗi lên màn hình, ta sử dụng hàm puts():

```
puts(str);
```

- Ngoài ta các hàm `printf("%s",str)` và `scanf("%s",str)` cũng có thể được sử dụng. Sinh viên tự thử sử dụng các hàm này và so sánh với các hàm gets() và puts().

- Về cơ bản, chuỗi ký tự được xem là mảng các ký tự nên các thao tác xử lý trên từng phần tử của mảng có thể áp dụng được trên chuỗi.

Ví dụ:

```
char str[50];
int n, i;
.....
n = strlen(str);
for(i=0; i<=n-1; i++)
     Xử lý ký tự thứ i str[i];
```

Bài 1: Viết chương trình nhập vào 1 chuỗi từ bàn phím. Hãy:

a. Viết hàm cho biết chuỗi đầu vào có là "PALINDROME" hay không? Nếu có kết quả trả về là 1, ngược lại kết quả là 0.

Gợi ý: Do chuỗi đối xứng là chuỗi khi đảo ngược sẽ giống hoàn toàn với chuỗi gốc nên ta có thể xét dựa vào so sánh lần lượt các phần tử đầu chuỗi với các phần tử cuối chuỗi.

Ví dụ: s = "abba"

Ta có s[0] == s[3]; s[1] == s[2] nên đó là chuỗi PALINDROME

Như vậy ta cần **biến i chạy từ 0 đến < (độ dài chuỗi) chia 2** và so sánh s[i] với s[**(độ dài chuỗi) -i-1**] nếu tồn tại 1 chỗ mà 2 ký tự khác nhau ta có thể kết luận ngay không đối xứng.

b. Viết hàm cho phép đảo ngược chuỗi đầu vào.

c. Viết hàm main() kiểm tra việc thực thi các hàm trên.

Bài 2: Tí đang tìm hiểu một trong những kĩ thuật mật mã hóa đơn giản nhất. Với bản tin cần được mã hóa, phương pháp này được sẽ mã hóa thành một chuỗi với quy tắc như sau:

- Chuỗi mã hóa chỉ lưu lại các ký tự chữ cái xuất hiện đầu tiên. Nói cách khác, các ký tự nào xuất hiện > 1 lần trở lên, sẽ bị xóa bỏ, chỉ giữ lại ký tự đầu tiên.

- Giá trị mật mã của chuỗi được tính bằng tổng số lần xuất hiện của các ký tự có tần suất > 1 lần.

Đầu vào bài toán là chuỗi nhập từ bàn phím. Đầu ra là 1 dòng duy nhất trong đó ghi giá trị mật mã của chuỗi, và chuỗi sau khi được mã hóa.

Ví dụ:

Dai hoc Can tho

→ 6 daihocnt

i am a man

→ 5 iamn

Gợi ý:

- Lưu ý các câu : "các ký tự nào xuất hiện > 1 lần trở lên, sẽ bị xóa bỏ, chỉ giữ lại kí tự đầu tiên.", "tần suất > 1 lần.",..

- Bài này nó nhập in hay thường thì đều phải chuyển về chữ thường.

- Lưu ý về khoảng trắng.

Bài 3: Viết chương trình nhập vào 1 chuỗi biểu diễn họ và tên của 1 người. Hãy:

a. Viết các hàm cho phép cắt bỏ các khoảng trắng không cần thiết trong chuỗi đầu vào. Tức là cắt bỏ hết tất cả các khoảng trắng ở đầu, ở cuối chuỗi; riêng giữa các từ chỉ giữ lại 1 khoảng trắng.

Gợi ý cắt bỏ khoảng trắng 2 đầu chuỗi bất kỳ

- Cắt khoảng trắng ở cuối chuỗi:
  - Bắt đầu từ cuối chuỗi
  - Dò ngược lên đầu chuỗi cho đến khi gặp ký tự != khoảng trắng
  - Đặt ký tự phía sau vị trí hiện tại là ký tự kết thúc chuỗi

- Cắt khoảng trắng ở đầu chuỗi:
  - Bắt đầu từ đầu chuỗi
  - Dò ngược về cuối chuỗi cho đến khi gặp ký tự != khoảng trắng
  - Trả về chuỗi từ vị trí hiện tại đến cuối chuỗi.

Sinh viên có thể viết thêm hàm **Delete(str, pos, num)** cho phép xóa trong chuỗi str từ vị trí pos, num ký tự; sau đó sử dụng hàm này để cắt khoảng trắng đầu, cuối và giữa.

b. Viết hoa ký tự đầu tiên của mỗi từ trong tên, các ký tự còn lại là ký tự thường.

c. Tách tên ra khỏi họ và tên.

Gợi ý

- Bắt đầu từ cuối chuỗi
- Dò ngược lại cho đến khi gặp ký tự khoảng trắng
- Từ ký tự sau vị trí hiện tại đến cuối chuỗi là tên cần tìm

d. Viết hàm main() để gọi thực thi các hàm trên.

Bài 4: Viết chương trình nhập vào 1 chuỗi ký tự. Giả sử không có khoảng trắng ở đầu ở cuối chuỗi và giữa các từ chỉ có 1 khoảng trắng. Hãy:

a. Tách các từ trong chuỗi ghi vào trong 1 mảng mà mỗi phần tử của là một chuỗi ký tự. Khai báo hàm này có thể như sau:

void TachTu(char str[], char Tu[][50], int *pN) ; // Tách chuỗi đầu vào lưu trong mảng Tu; pN là con trỏ chỉ tới vùng nhớ chứa số từ trong chuỗi.

b. Cho biết mỗi từ xuất hiện bao nhiêu lần?

Ví dụ: Chuỗi nhập: Hoc Tin Hoc o Dai hoc Can Tho

→ hoc: 3
tin: 1
o: 1
dai: 1
can: 1
tho: 1

Chương 9: KIỂU CẤU TRÚC

Một chương trình cần phải lưu trữ thông tin của một nhân viên gồm mã nhân viên, họ tên, ngày sinh, giới tính, lương. Để thực hiện ta có thể khai báo các biến để lưu trữ các thông tin này:

- `char id[8]; // B1700908`
- `char name[50]; // Nguyen Van Minh`
- `char dob[10];// 01/01/1998`
- `char gender; // M`
- `float salary; // 1500.00f`

Nhận xét:

- Khó quản lý khi có nhiều biến, chương trình lớn.
- Truyền tham số cho hàm quá nhiều
- Tìm kiếm, sắp xếp, sao chép, … khó khăn

Ý tưởng giải quyết:

- Gom những thông tin của cùng 1 đối tượng thành một kiểu dữ liệu mới \Kiểu Cấu Trúc (**struct**)

Việc khai báo kiểu cấu trúc dữ liệu này trong C gồm đặc tả tên của cấu trúc và kiểu của mỗi phần tử (trường). Thông thường việc khai báo này có dạng:

```
struct <structure name>{
        type-1          list-1;
        type-2          list-2;
        …
        type-n          list-n;
};
```

Mỗi *list-i* là một danh biểu hay một danh sách các danh biểu của các trường của cấu trúc; *type-i* là kiểu các trường này.

Ví dụ, cấu trúc nhân viên bên trên có thể được khai báo như sau:

```
struct Employee{
     char id[8];
     char name[50];
     char dob[10];
     char gender;
     float salary;
};
```

Sau đó ta có thể khai báo bất kỳ biến nào có kiểu Employee:

```
struct Employee emp;
```

Trong ví dụ này, trường đầu tiên của **struct** là id là một chuỗi 8 ký tự, trường thứ 2 name là một chuỗi 50 ký tự, trường thứ 3 dob là một chuỗi 10 ký tự, trường thứ 4 gender là 1 ký tự và trường cuối cùng salary là số thực **float**. Một giá trị tiêu biểu của emp có thể như hình vẽ dưới đây:

id	name	dob	gender	salary
B1700908	Nguyen Van Minh	01/01/1998	M	1500.00f

Mỗi trường của cấu trúc có thể được truy cập trực tiếp bằng cách dùng biến chỉ trường dạng: <struct variable>.<field-name>.

Ví dụ:

```
gets(emp.id);
strcpy(emp.name,"Nguyen Van Minh");
emp.salary = 1500.00f;
```

Bài 1: Số phức là một cặp (a, b) trong đó a, b là các số thực, a gọi là phần thực, b là phần ảo. (Đôi khi người ta cũng viết số phức dưới dạng a + ib trong đó i là một đơn vị ảo có tính chất i^2=-1).

Gọi số phức c1=(a1, b1) và c2=(a2,b2) khi đó tổng của hai số phức c1 và c2 là một số phức c3 mà c3=(a1+a2, b1+b2).

Hãy khai báo kiểu dữ liệu số phức cho phù hợp, sau đó viết chương trình cho phép nhập từ bàn phím 2 số phức c1, c2; tính số phức tổng và hiển thị kết quả.

Phân tích:

1. Khai báo cấu trúc số phức gồm 2 trường: phần thực a và phần ảo b.
2. Viết hàm cho phép nhập từ bàn phím một số phức, kết quả trả về là số phức vừa nhập.
3. Viết hàm cho phép hiển thị 1 số phức đầu vào. Thực chất hàm này là hiển thị phần thực và phần ảo của số này.
4. Viết hàm tính tổng 2 số phức c1 và c2. Khai báo hàm này có thể là:

   ```
   void Tong(SoPhuc c1, SoPhuc c2, SoPhuc *pC);
   ```

   Trong đó pC là con trỏ chỉ đến vùng nhớ chứa số phức kết quả của phép cộng.
5. Viết hàm main() cho phép nhập từ bàn phím 2 số phức c1, c2. Goi hàm tính tổng và sau đó gọi hàm hiển thị 2 số phức đầu vào và số phức kết quả.

```c
#include<stdio.h>

struct SoPhuc
{
    float Thuc;
    float Ao;
};

SoPhuc Nhap()
{
    SoPhuc c;
    printf("Phan thuc: ");scanf("%f",&c.Thuc);
    printf("Phan ao: ");scanf("%f",&c.Ao);
    return c;
}

void HienThi(SoPhuc c)
{
    printf("%.2f + i%.2f\n",c.Thuc,c.Ao);
}

void Tong(SoPhuc c1, SoPhuc c2, SoPhuc *pC)
{
    pC->Thuc=c1.Thuc+c2.Thuc;
    pC->Ao=c1.Ao+c2.Ao;
}

int main()
{
    SoPhuc c1,c2,c;
    printf("Nhap so phuc thu nhat:\n");
    c1 = Nhap();
    printf("Nhap so phuc thu hai:\n");
    c2=Nhap();
    printf("So phuc thu nhat: ");
    HienThi(c1);
    printf("So phuc thu hai: ");
    HienThi(c2);
    Tong(c1,c2,&c);
    printf("Tong: ");
    HienThi(c);
    return 0;
}
```

Bài 2: Trong đa số ngôn ngữ lập trình, phân số không phải là kiểu dữ liệu có sẵn. Do đó để định nghĩa kiểu phân số, người ta dùng 1 cấu trúc có 2 trường: tử số và mẫu số (cả hai có kiểu là int). Hãy:

1. Khai báo kiểu dữ liệu phân số
2. Viết các hàm:
   a. Nhập tử số và mẫu số từ bàn phím. Kết quả trả về là phân số có tử và mẫu vừa nhập.
   b. Hiển thị một phân số A lên màn hình dạng: <Tử số>/<Mẫu số>
   c. Tối giản 1 phân số A. Chẳng hạn với phân số 3/6 thì dạng tối giản là 1/2. Khai báo hàm này có thể là:

      void ToiGian(PhanSo *pA);

      Với pA là con trỏ chỉ tới 1 phân số cần phải tối giản.

      Gợi ý: - Tìm ước chung lớn nhất (UCLN) của tử số và mẫu số.

      - Tử số và mẫu số mới được tính bằng cách chia chúng cho UCLN.

   d. Tính tổng 2 phân số A và B

      Gợi ý: - Tìm mẫu số chung của 2 mẫu số; mẫu số chung này là bội chung nhỏ nhất của chúng.

      - Quy đồng mẫu số và tính tổng.

      - Trả về kết quả

   e. Tính hiệu của 2 phân số A và B

      Gợi ý: Như câu d

   f. Tính tích 2 phân số A và B
   g. Tính kết quả của phép chia phân số A cho B.
3. Viết hàm main() để kiểm chứng việc thực thi các hàm trên.

Bài 3: Hãy định nghĩa kiểu:

```
struct Hoso{
    char HoTen[40];
    float Diem;
    char Loai[10];
};
```

1. Viết hàm nhập vào họ tên, điểm của n học sinh; đồng thời xếp loại theo điểm như sau:

Điểm	**Xếp loại**
≥9.0	Xuất sắc
≥8.0 và <9.0	Giỏi
≥6.5 và < 8.0	Khá
≥5.0 và <6.5	Trung bình
≥4.0 và <5.0	Yếu
< 4.0	Kém

2. Hiển thị thông tin của n học sinh
3. Sắp xếp danh sách n học sinh giảm dần theo điểm. Hiển thị kết quả sau khi sắp xếp.

Bài 4: Viết chương trìnhquản lý sinh viên:

1. Khai báo cấu trúc sinh viên gồm các trường: MSSV, HỌ TÊN, NGÀY SINH, QUÊ QUÁN, ĐIỂM TRUNG BÌNH TÍCH LUỸ, ĐIỂM RÈN LUYỆN.
2. Khai báo mảng cấu trúc để quản lý 1 lớp gồm 50 sinh viên.
3. Nhập thông tin của n sinh viên.
4. In thông tin n sinh viên.
5. In thông tin sinh viên thứ n.
6. Tìm thông tin sinh viên theo MSSV.
7. Tìm sinh viên có ĐTBTL/ ĐRL cao nhất.
8. Xếp hạng sinh viên theo ĐTBTL

Chương 10: TẬP TIN

TẬP TIN là công cụ cho phép lưu trữ dữ liệu lâu dài (kể cả khi chương trình kết thúc).

Về cơ bản, có 2 loại tập tin:

1. Tập tin văn bản: là loại tập tin dùng để lưu trữ các ký tự dưới dạng mã ASCII của chúng. Điểm đặc biệt là dữ liệu của tập tin được mô tả thành các dòng, mỗi dòng được kết thúc bằng ký tự xuống dòng (new line), ký hiệu '\n'; ký tự này là sự kết hợp của 2 ký tự CR (Carriage Return - Về đầu dòng, mã ASCII là 13) và LF (Line Feed - Xuống dòng, mã ASCII là 10).

Mỗi tập tin được kết thúc bởi ký tự EOF (End Of File) có mã ASCII là 26 (xác định bởi tổ hợp phím Ctrl + Z).

2. Tập tin nhị phân: là loại tập tin lưu trữ các byte dữ liệu. Loại tập tin này có thể là tập tin định kiểu hoặc không định kiểu.

Con trỏ tập tin: Khi một tập tin được mở ra để làm việc, tại mỗi thời điểm, sẽ có một vị trí của tập tin mà tại đó việc đọc/ghi thông tin sẽ xảy ra. Người ta hình dung có một con trỏ đang chỉ đến vị trí đó và đặt tên nó là con trỏ tập tin.

Sau khi đọc/ghi xong dữ liệu, con trỏ sẽ chuyển dịch thêm một phần tử về phía cuối tập tin. Sau phần tử dữ liệu cuối cùng của tập tin là dấu kết thúc tập tin EOF (End Of File).

Các bước thao tác với tập tin trong lập trình C

- Khai báo biến tập tin.
- Mở tập tin bằng hàm fopen().
- Thực hiện các thao tác xử lý dữ liệu của tập tin bằng các hàm đọc/ghi dữ liệu.
- Đóng tập tin bằng hàm fclose().

Ở đây, ta thao tác với tập tin nhờ các hàm được định nghĩa trong thư viện **stdio.h**.

Bài 1: Mảng đối xứng gọi là PALINDROME. Ví dụ mảng 1.0 2.0 3.0 2.0 1.0 là PALINDROME. Viết chương trình cho phép kiểm tra một mảng có là PALINDROME hay không ? Nếu có kết quả là YES, ngược lại kết quả là NO.

Dữ liệu đầu vào của bài toán được cho trong tập tin văn bản B9_Bai1.inp gồm n số thực mỗi số cách nhau 1 khoảng trắng.

Đầu ra của bài toán được ghi lên tập tin B9_Bai.out gồm giá trị YES nếu mảng số thực trong tập tin đầu vào là PALINDROME và NO nếu không là PALINROME.

Đối với bài này để đọc dữ liệu từ tập tin văn bản ta khai báo 1 mảng 1 chiều A và dùng 1 vòng lặp cho tới khi EOF, mỗi lần đọc 1 số thực (nhờ fscanf) và lưu vào mảng A.

```
void readFile(char fileName[], float A[], int *pN)
{
    FILE *f;
    f = fopen(fileName,"rt");
    if (f!=NULL)
    {
        *pN=0;
        while(!feof(f))
        {
            fscanf(f,"%f",&A[*pN]);
            (*pN)++;
        }
        fclose(f);
    }
}
```

Chương trình này ghi kết quả là YES hoặc NO lên tập tin kết quả ; do đó ta có thể thiết kế 1 hàm ghi lên tập tin như sau :

```
void writeFile(char fileName[], char Result[])
{
    FILE *f;
    f = fopen(fileName,"wt");
    if (f!=NULL)
    {
        fprintf(f,"%s",Result);
        fclose(f);
    }
}
```

Hàm kiểm tra 1 mảng có là PALINDROME hay không, kết quả trả về là 1 nếu phải, ngược lại kết quả trả về 0 và hàm main() cho phép thực thi chương trình :

```
int checkPalindrome(float A[], int n)
{
    int k = 1;
    int i;
    for(i=0;i<n/2;i++)
        if (A[i] != A[n-i-1]) k=0;
    return k;
}

int main()
{
    float A[100];
    int N;
    readFile("B9_Bai1.inp",A,&N);
    if (checkPalindrome(A,N))
        writeFile("B9_Bai1.out","YES");
    else
        writeFile("B9_Bai1.out","NO");
    return 0;
}
```

Bài 2: Cuối mỗi tháng, một bản tin được tạo ra để chỉ tình trạng về tài khoản của người dùng trong tập tin văn bản UsersFile (UsersFile.txt, mỗi trường cách nhau ký tự TAB).

Số hiệu	Tên người dùng	Giới hạn	Đã dùng
100106	Nguyen Huu Thang	750	332.12
100107	Miura Toshiya	850	337.43

1. Khai báo cấu trúc người dùng
2. Viết hàm đọc từ tập tin UsersFile.txt vào 1 mảng là danh sách người dùng, con trỏ *pN chỉ đến vùng nhớ chứa số phần tử của mảng.

Giải thuật để đọc từ tập tin trong trường hợp này có thể là

Input : Tập tin UsersFile.txt

Output: Mảng A có con trỏ pN chỉ tới số phần tử của mảng A

```
FILE *f = fopen("UsersFile.txt","rt");
if (f!=NULL){
      char str[255];
      int i=0;
      while(!feof(f)){
            memset(str,'\0',255);
            fgets(str, 255, f);
            Phân giải chuỗi str lưu vào trong phần tử A[i];
            i++;
      }
      (*pN) = i;
      fclose(f);
}
```

Giải thuật phân giải chuỗi str và lưu từng trường vào cấu trúc Người dùng

Input: Chuỗi str chứa các trường của cấu trúc người dùng mỗi trường cách nhau ký tự TAB

Ouput: Con trỏ chỉ cấu trúc người dùng pU

```
char sID[7];
char sName[30];
char sLimit[12];
char sUsed[20];
int j;
int i=0;
//Get ID
while(i<strlen(Line) && Line[i]!=TAB) i++;
strncpy(sID, Line, i);
sID[i]='\0';
```

Chuyển sID từ chuỗi thành số nguyên và số hiệu User của con trỏ pU là số này

//Get Name

```
i++; j=i;
while(i<strlen(Line) && Line[i]!=TAB) i++;
strncpy(sName, Line+j, i-j);
sName[i-j] = '\0';
```

Tên User của con trỏ pU là sName;

//Get Limit

```
i++; j=i;
while(i<strlen(Line) && Line[i]!=TAB) i++;
strncpy(sLimit, Line+j, i-j);
sLimit[i-j]='\0';
```

Chuyển sLimit thành số nguyên và cho giới hạn của con trỏ pU là số này

//Get Used

```
j=i+1;
strncpy(sUsed,Line+j,strlen(Line)-j);
sUsed[strlen(Line)-j] ='\0';
```

Chuyển sUsed thành số thực và cho tài khoản đã dùng của con trỏ pU là số này

3. Viết hàm liệt kê danh sách các người dùng vừa đọc.
4. Liệt kê những người dùng đã sử dụng 90% hay hơn số tài khoản có thể dùng của anh ta.
5. Sắp xếp danh sách các người dùng tăng dần theo số hiệu người dùng
6. Ghi danh sách đã sắp xếp lên tập tin văn bản kết quả UserFile.out.

Giải thuật để ghi lên tập tin trong trường hợp này có thể là

Input : Mảng người dùng A có n phần tử được sắp thứ tự tăng dần theo số hiệu người dùng

Output: Tập tin UsersFile.out mỗi người dùng trên 1 dòng, mỗi trường của người dùng cách nhau ký tự TAB

```
FILE *f = fopen("UsersFile.out","wt");
if (f!=NULL){
      for(i=0;i<=N-1;i++)
```

Dùng fprintf để ghi Số hiệu, tên người dùng, giới hạn, tài khoản sử dụng của người dùng A[i] lên tập tin chỉ bởi con trỏ f; mỗi giá trị cách nhau ký tự TAB

```
      close(f);
}
```

7. Ghi danh sách đã sắp xếp lên tập tin kết quả UserFile.dat là tập tin gồm các cấu trúc Người dùng (tập tin nhị phân)

Giải thuật để ghi lên tập tin nhị phân UsersFile.dat như sau

Input: Mảng người dùng A có n phần tử được sắp thứ tự tăng dần theo số hiệu người dùng

Output: Tập tin nhị phân UsersFile.dat trong đó thông tin mỗi người dùng ghi liên tiếp nhau

```
FILE *f = fopen("UsersFile.dat","wb");
if (f!=NULL){
  for(i=0;i<=N-1;i++)
  {
      Ghi số hiệu của người dùng lên tập tin chỉ bởi con trỏ f (fwrite)
      Ghi chiều dài của họ tên người dùng lên tập tin chỉ bởi con trỏ f (fwrite)
      Ghi họ tên người dùng lên tập tin chỉ bởi con trỏ f (fwrite)
      Ghi giới hạn của người dùng lên tập tin chỉ bởi con trỏ f (fwrite)
      Ghi tài khoản sử dụng của người dùng lên tập tin chỉ bởi con trỏ f (fwrite)
  }
  fclose(f);
}
```

Bài 3: Viết chương trình đọc nội dung tập tin UsersFile.dat ở bài 2 và hiển thị danh sách người dùng trong tập tin này lên màn hình.

Gợi ý:

- Khai báo cấu trúc người dùng

- Đọc dữ liệu từ tập tin nhị phân UsersFile.dat vào mảng người dùng A có con trỏ pN chỉ đến số phần tử như sau:

Input: Tập tin nhị phân UsersFile.dat trong đó thông tin mỗi người dùng ghi liên tiếp nhau

Output: Mảng người dùng A có con trỏ pN chỉ đến số người dùng trong tập tin dữ liệu

```
FILE *f = fopen("UsersFile.dat","rb");
if (f!=NULL){
      int i=0;
      while(!feof(f)){
            Khai báo người dùng u
            Cấp phát vùng nhớ chứa họ tên của u (malloc)
            Đọc số hiệu của người dùng u từ tập tin chỉ bởi f (fread)
            Đọc chiều dài chuỗi của họ tên người dùng từ tập tin chỉ bởi f (fread)
```

Đọc họ tên người dùng u từ tập tin chỉ bởi f (fread)

Đọc giới hạn người dùng u từ tập tin chỉ bởi f (fread)

Đọc tài khoản sử dụng của người dùng u từ tập tin chỉ với f (fread)

```
            A[i] = u;
            i++;
        }
        (*pN) = i;
        fclose(f);
    }
```

Chương 11: BÀI TẬP TỔNG HỢP 2

Bài 1: **Mảng thứ tự tăng**

Cho một mảng n số nguyên. Kiểm tra xem mảng số nguyên này có được sắp thứ tự tăng dần hay không?

Bài làm	**tangdan.c**
Dữ liệu vào	***tangdan.inp*** - Dòng 1: 1 số nguyên n - Dòng kế tiếp ghi n giá trị nguyên biểu diễn n phần tử của mảng, mỗi giá trị cách nhau 1 khoảng trắng.
Dữ liệu ra	***tangdan.out*** CO nếu mảng được sắp tăng dần; KHONG nếu ngược lại

Ví dụ 1:	
tangdan.inp	tangdan.out
5 -7 0 0 4 10	CO
Ví dụ 2:	
tangdan.inp	tangdan.out
6 0 3 -6 10 12 15	KHONG

Đọc dữ liệu từ tập tin vào

Để đọc dữ liệu từ tập tin này ta đọc dòng đầu tiên để ghi nhận số phần tử n của mảng, sau đó là một vòng lặp để lặp đúng n lần, mỗi lần đọc 1 giá trị của mảng đã cho.

Input: tangdan.inp

Output: Mảng A có con trỏ pN chỉ đến số phần tử của nó

```
FILE *f = fopen("tangdan.inp","rt");
if (f!=NULL){
        Dùng fscanf để đọc số phần tử của mảng cho con trỏ pN
        Cấp phát 1 vùng nhớ có (*pN) phần tử, cho A là địa chỉ bắt đầu của vùng nhớ này
        for(i=0;i<=(*pN)-1;i++)
              Đọc phần tử thứ I của mảng A nhờ fscanf
        fclose(f);
}
```

Kiểm tra mảng A tăng dần: Ta giả sử ban đầu mảng tăng dần (cho biến k=1); sau đó lặp từ đầu tới cuối của mảng A, nếu có 2 phần tử kế nhau không đúng thứ tự (phần tử trước > phần tử sau)→ mảng không tăng dần (tức k=0).

Bài 2: Đếm số nguyên âm trong chuỗi. Viết chương trình đếm số lượng nguyên âm trong 1 chuỗi cho trước (có tối đa 255 ký tự). Biết nguyên âm gồm những ký tự ‘a’, ‘e’, ‘o’, ‘u’, ‘i’ (không phân biệt hoa thường).

Bài làm	**nguyenam.c**
Dữ liệu vào	***nguyenam.inp*** 1 dòng duy nhất là chuỗi ký tự kết thuc bằng ký tự xuống dòng
Dữ liệu ra	***nguyenam.out*** 1 số nguyên chỉ số lượng nguyên âm trong chuỗi

Ví dụ 1:	
nguyenam.inp	nguyenam.out
Toi di hoc o Can THO	7
Ví dụ 2:	
nguyenam.inp	nguyenam.out
DHCT	0

Bài 3: **Trò chơi Puzzle**

Cho một bảng hình chữ nhật kích thước *m*n* (*m* dòng, *n* cột), các dòng được đánh thứ tự từ trên xuống, các cột được đánh thứ tự từ trái sang. Có một ô trong bảng không nhận giá trị (ô trống), các ô còn lại nhận 1 giá trị từ *1..m*n-1* (không có 2 ô có giá trị giống nhau).

Người chơi sẽ tiến hành hoán đổi ô trống và các ô có giá trị lân cận để đạt đến trạng thái kết thúc của trò chơi.

Trò chơi gọi là kết thúc nếu các giá trị *1..m*n-1* lần lượt theo thứ tự được bố trí vào các ô (1, 1), (1, 2), …, (1,n); (2, 1), (2, 2), …,(2, n),…; (m,1), (m,2),…, (m, n-1).

Cho kích thước của bảng chữ nhật *m*n* và giá trị hiện hành của các ô trong bảng. Kiểm tra xem trò chơi đạt trạng thái kết thúc hay chưa?

Bài làm	**puzzle.c**
Dữ liệu vào	***puzzle.inp*** - Dòng 1: 2 số nguyên m và n cách nhau 1 khoảng trắng - m dòng tiếp theo mỗi dòng ghi n giá trị (từ *0..m*n-1*), mỗi giá trị cách nhau 1 khoảng trắng. Giá trị *0* tại ô (i, j) cho biết ô (i,j) là ô trống.
Dữ liệu ra	***puzzle.out*** CO nếu là trạng thái kết thúc; KHONG nếu chưa là trạng thái kết thúc

Ví dụ 1:	
puzzle.inp	puzzle.out
3 3 1 2 3 6 5 4 7 8 0	KHONG
Ví dụ 2:	
puzzle.inp	puzzle.out
2 3 1 2 3 4 5 0	CO

Chương 12: BÀI TẬP TỔNG HỢP 3

Bài 1: **Mảng các số lẻ**

Cho một mảng n số nguyên. Kiểm tra xem mảng số nguyên này có tất cả các số đều là số lẻ hay không?

Bài làm	**mangle.c**
Dữ liệu vào	mangle.inp - Dòng 1: 1 số nguyên n - Dòng kế tiếp ghi n giá trị nguyên biểu diễn n phần tử của mảng, mỗi giá trị cách nhau 1 khoảng trắng.
Dữ liệu ra	mangle.out CO nếu mảng gồm các số lẻ; KHONG nếu ngược lại

Ví dụ 1:	
mangle.inp	mangle.out
5 -7 1 -1 5 11	CO
Ví dụ 2:	
Mangle.inp	mangle.out
6 0 3 -6 10 12 15	KHONG

Bài 2: **Độ lệch chuẩn**. Cho trước bảng số liệu về chiều cao của 1 quần thể. Tính độ lệch chuẩn của bộ số liệu đó.

Độ lệch chuẩn cho ta biết về sự biến thiên, từng giá trị quan sát có mối liên hệ tập trung như thế nào xung quanh giá trị trung bình.

- Nếu độ lệch chuẩn bằng 0 => phương sai bằng 0 => các giá trị quan sát cũng chính là giá trị trung bình hay nói cách khác không có sự biến thiên nào cả.

- Nếu độ lệch chuẩn càng lớn => sự biến thiên xung quang giá trị trung bình càng lớn.

Cách tính độ lệch chuẩn - Standard deviation (SD) = $\sqrt{\frac{\sum_{1}^{n}(X_i - \overline{X})^2}{n-1}}$

Để tính độ lệch chuẩn ta cần xác định giá trị sau:

- Giá trị trung bình $\overline{X}$

- Phương sai của bộ số liệu SD^2

Bước 1: Tính giá trị trung bình của bộ số liệu

Giá trị trung bình bằng trung bình cộng các giá trị của tất cả bộ số liệu hay chính bằng tổng các giá trị trong bộ số liệu chia cho tổng số các giá trị có trong bộ số liệu.

Bước 2: Tính phương sai của bộ số liệu

Phương sai là giá trị đặc trưng cho độ phân tán (biến thiên) của các số liệu trong bộ số liệu so với giá trị trung bình của bộ số liệu.

Bài làm	**dolechchuan.c**
Dữ liệu vào	dolechchuan.inp - Dòng 1: 1 số nguyên n - Dòng kế tiếp ghi n giá trị nguyên biểu diễn n chiều cao của quần thể, mỗi giá trị cách nhau 1 khoảng trắng.
Dữ liệu ra	dolechchuan.out Giá trị độ lệch chuẩn với 2 số lẻ thập phân

Ví dụ:	
dolechchuan.inp	dolechchuan.out
5 142 150 187 180 145	21.06